HONEYWELL SOFTWARE ENGINEERING TEAMS — IA · BA · PA&T

AI Champions Programme — GitHub Copilot Enterprise, SDD, MCP & Agentic Software Engineering

Module 09

Studio-Style No-Code Agents (Rovo Agent Studio)

Let product, program and delivery roles shape agent behaviour without code: build a grounded Rovo agent, wire it into an automation, add human checkpoints, and know where a studio workflow hands over to engineering.

DURATION
2 hours

FORMAT
Concepts · build lab

PRIMARY TOOL
Atlassian Rovo Studio

How We'll Spend These Two Hours

From what Studio is, to a governed agent and automation, to the handover to engineers.

01

Rovo Studio & Building Blocks

15 min

Agents, automation flows and apps, and who builds them.

02

Authoring a Grounded Agent

25 min

Identity, instructions, knowledge sources and skills, with role-based use cases.

03

Automations & Approvals

20 min

Triggers, the agent step, actions, and human-in-the-loop checkpoints.

04

Governance & Handover

15 min

Permission-aware access, controls, and the studio-versus-engineering boundary.

05

Lab: Build, Test, Hand Over

40 min

Build an agent plus automation on sample Jira/Confluence data and hand it to an owner.

06

Best Practices & Wrap-up

5 min

Do and avoid, then the step up to code-based agents in Module 10.

Module 09 in the 12-Module Journey

Same agent ideas, a different audience: business and program roles authoring agents in a studio.

FOUNDATION			BUILD & VALIDATE			EXTEND WITH TOOLS & AGENTS				GOVERN	CAPSTONE
1	2	3	4	5	6	7	8	**9**	10	11	12
Objectives & GenAI Recap	Copilot Advanced	Prompt & Context Eng.	Spec-Driven Dev (Spec Kit)	SDD to Full SDLC	Test Strategy & Validation	MCP & Custom MCP Server	PR Automation & LLM-Judge	**No-Code Agents (Rovo)**	Code Agents & Simple RAG	Observability, ROI & Gov.	Capstone & Roadmap

M03
Good agent instructions are prompt and context engineering

M07
MCP and studio connectors both give agents reach — and need limits

M08
Human checkpoints and judging apply to studio agents too

M10
The same agent idea, now authored in code with the Copilot SDK

Why this matters: the people closest to the work (program managers, product owners, delivery leads) can now shape agents directly. That needs guardrails, not gatekeeping.

Rovo Studio: Three Building Blocks, No Code by Default

Describe the problem in plain language, or build each piece by hand.

Agents

Answer from your organisation's knowledge, follow multi-step scenarios and take actions in Jira and Confluence.

You define identity, instructions, knowledge and skills

Automation flows

React to events, check conditions and run actions across Atlassian and third-party apps.

Triggers, branches, conditions, actions

Apps

No-code apps that extend Jira and Confluence and can use agents, flows and skills.

Can later move to code through Forge

WHO BUILDS WHAT

Makers & power users

Admins & team leads

Support, IT, marketing builders

Needs to be true: Studio requires Rovo AI. If your organisation has blocked Rovo, users can open Studio but cannot build anything — check this first.

Anatomy of a Studio Agent

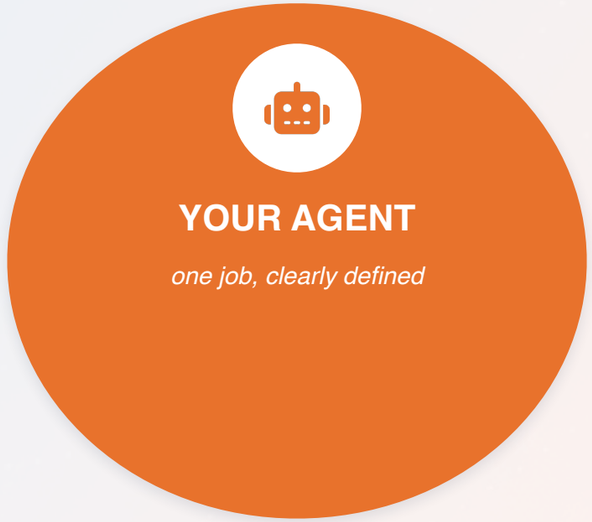
Four settings decide what the agent knows, how it behaves and what it can do.

Identity

Name, role and who it serves

Instructions

Job, inputs, outputs, guardrails



Knowledge sources

Spaces, pages, projects it may use

Skills & actions

Tools it can use to act in apps

Rule: out-of-the-box agents do not take actions. A custom agent can only act through the skills you give it — which is also your first safety control.

Writing Instructions: A Six-Part Template

Specific instructions beat clever ones. Say what to do, for whom, and what to avoid.

1 — Name & role
One-line job

2 — Target users
Who it serves

3 — Job to be done
From the user's view

4 — Inputs
What the user supplies

5 — Outputs
Format of the result

6 — Guardrails
What to avoid; how to say "I don't know"

VAGUE

"Help with project status. Be accurate and professional."

No audience, no sources, no format, no limits.

SPECIFIC (illustrative)

"You are the Release Status Agent for programme leads. Using only the Release space, list open blockers per release as a table with owner and link. If a fact is not in the space, say so — do not guess."

Tip: ask Rovo to draft the first version from your description, then edit it against the six parts. Test, change one thing, test again.

Grounding: Knowledge Sources and Permission-Aware Access

Narrow what the agent can read — and remember it only shows what the asking user may see.

Person asking

Their permissions travel with the question

Agent

Reads only its chosen knowledge scope

Release space (Confluence)	In scope
Programme project (Jira)	In scope
HR space	Out of scope
Finance space	Out of scope

Pick sources

Spaces, parent pages, projects — not “everything”

Respect access

Users never see content they cannot already view

Check web search

Public web only if an admin has enabled it

Accuracy tip: narrower knowledge usually means more focused answers. Start with one space and widen only when you have a reason.

Real-World Agents by Role: Who Builds What, for What Job

Pick a narrow job with a clear owner. Each agent does the legwork; a person keeps the decision.

Programme reporting

Programme manager

AGENT DOES

Summarises status from Jira and Confluence into a standard weekly format.

HUMAN DECIDES

Reviews and sends the report

Issue triage

Product owner / support lead

AGENT DOES

Reads new work items, suggests type, priority and owner, and drafts a first reply.

HUMAN DECIDES

Confirms or changes the suggestion

Release coordination

Delivery lead

AGENT DOES

Collects open blockers and checklist gaps per release from linked pages and issues.

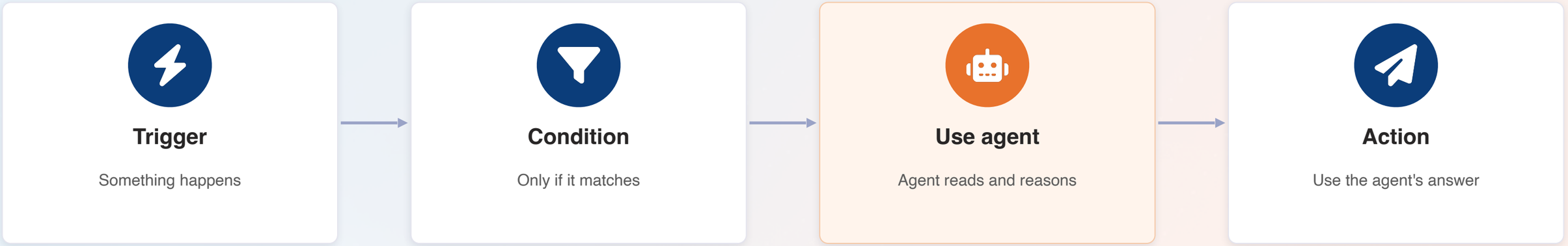
HUMAN DECIDES

Decides go / no-go

Choosing a first agent: pick a job that is frequent, text-heavy, low-risk and easy to check. Avoid anything where a wrong answer is expensive or hard to undo.

From Agent to Automation: Work Starts on a Trigger

An automation flow adds the “when” and the “then”; the agent supplies the thinking in the middle.

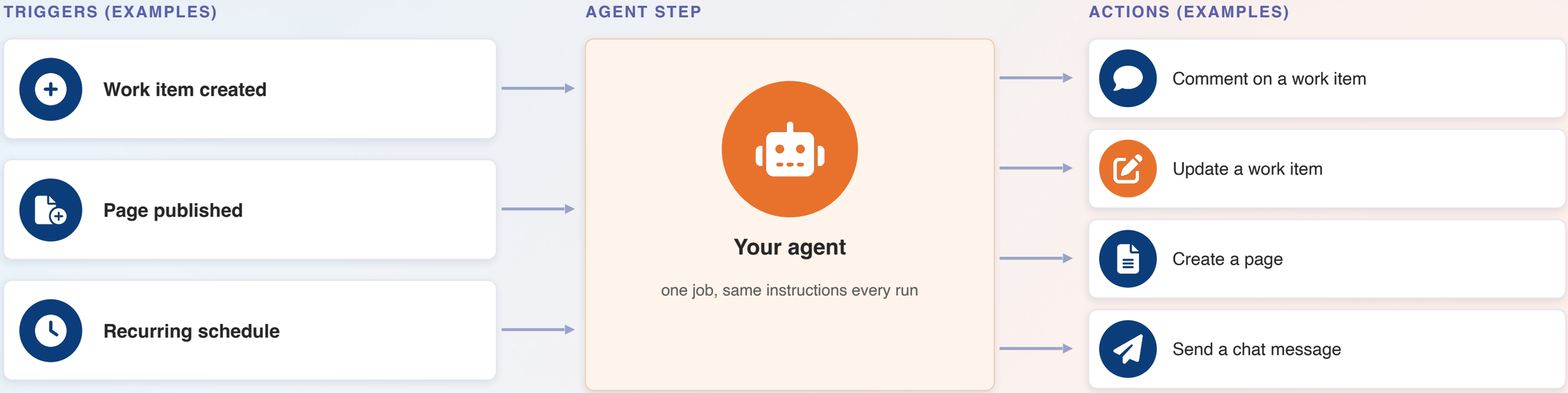


How the result travels: the agent's response is available to later steps as the `{{agentResponse}}` smart value — for example, to post a comment or send a message.

Two ways in: add an automation trigger from the agent's own settings, or add a “use agent” step inside a flow. Either way, finish the flow in Automations.

Triggers In, Actions Out

The same agent can serve many flows. What changes is the trigger and what you do with the answer.



Know the limits: custom agents need a skill for each action they perform, and automations time out after 15 minutes — so keep the agent's job small enough to finish.

Three Flow Recipes You Can Build This Afternoon

Each follows the same shape: trigger → agent → draft → human checkpoint.

Issue triage

New work item → Agent suggests type + owner → Comment as a suggestion → Owner confirms

Weekly programme status

Every Monday → Agent drafts summary from sources → Create draft page → Programme lead edits + publishes

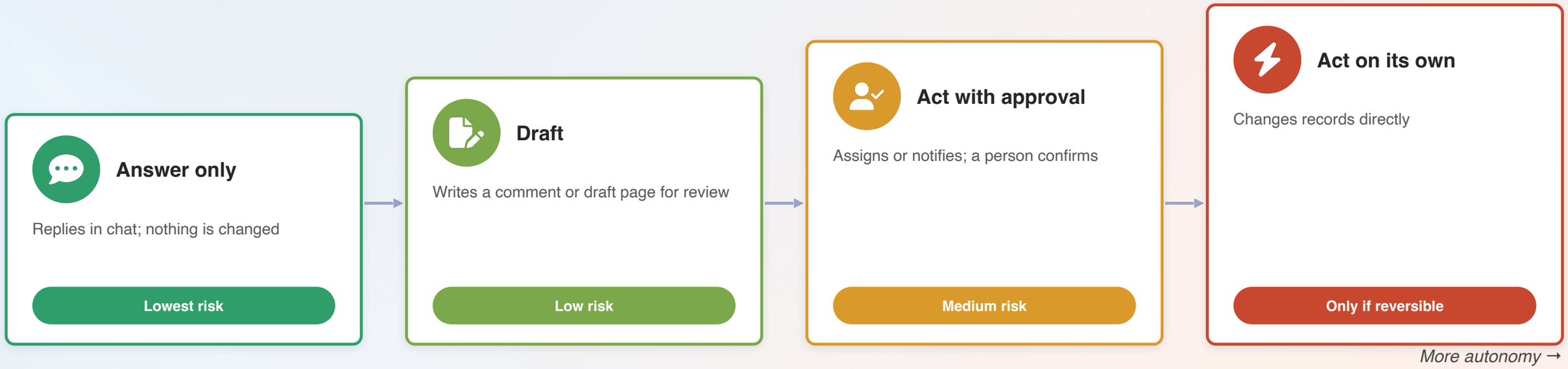
Release readiness

Release page published → Agent lists open blockers → Post to the release chat → Delivery lead decides

Notice: in every recipe the agent's output lands as a suggestion or draft, and a named person takes the final step. That is the checkpoint — build it in on purpose.

Human-in-the-Loop: Choose How Much Freedom the Agent Gets

Agents in automations can act without a per-action confirmation — so the checkpoint is something you design.



Start low: begin at “Draft” and move right only when test results and a named owner justify it. Never start at “Act on its own” with records that are hard to restore.

Governance: Risks and Their Controls

Most controls are choices you make while building — plus switches your admins hold.

RISK	CONTROL
Agent reads more than intended	Narrow knowledge sources; start with one space
Agent acts in ways nobody expected	Give only the skills the job needs; admins can block agents acting in automations
A wrong answer is published	Draft output and a named reviewer before anything is posted
Flow fails or times out	Keep jobs small; automations time out after 15 minutes
Nobody owns it	Name an owner, document purpose and review on a schedule

Permission model: an agent respects the permissions of the person who prompts it. In automations, agents act on your behalf — confirm whose permissions a flow uses before you rely on it.

The Boundary: Studio Workflow or Engineering Agent?

Studio is the fast path. Hand over when the job outgrows what a studio should carry.

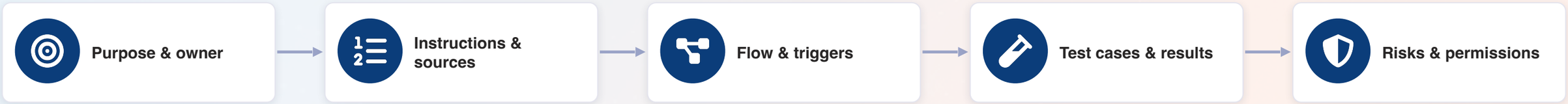
STAY IN STUDIO WHEN...

- The job lives in Jira and Confluence data
- A person reviews the output
- Rules fit triggers, conditions and a few actions

HAND TO ENGINEERING WHEN...

- It needs custom code, tests or other systems
- It must run at scale or with strict reliability
- It needs retrieval or tools beyond the studio

THE HANDOVER PACK



Bridge to Module 10: the same agent idea — instructions, knowledge, tools — can be rebuilt in code with the Copilot SDK when the job needs it.

Lab: Build, Test, Hand Over

LAB 1 · 40 min

Build a grounded agent and an automation on sample data, then hand it to a technical owner.

Goal Create a Rovo agent and one automation flow for a real-world scenario, test it on representative data, and write the handover.

STEPS

1. Choose a scenario (programme reporting, issue triage or release coordination). Write the six-part instructions and ground the agent on one sample space or project.
2. Test with five prompts, including one out of scope and one the sources cannot answer. Change one thing at a time and re-test.
3. Build the flow: trigger, agent step, and an action that writes a draft or suggestion. Add the human checkpoint and name the reviewer.
4. Run it on sample data. Write handover.md so a technical owner could take it over or rebuild it in code.

FILES YOU CREATE

repo/

```
lab09/
  agent-instructions.md
  test-cases.md
  flow-notes.md
  handover.md
```

DONE WHEN

The flow runs on sample data, the agent declines the out-of-scope prompt, and handover.md covers purpose, owner, sources, risks and test results.

Best Practices for Studio Agents and Automations

Small jobs, narrow knowledge, drafts first, named owners.

DO THIS	AVOID THIS
Give each agent one clear job and a named owner	Build one “do everything” agent for the whole team
Write instructions with all six parts, including guardrails	Leave knowledge scope at “everything” by default
Narrow knowledge sources to what the job needs	Let an agent change records unreviewed on day one
Output drafts or suggestions; a person takes the last step	Write vague instructions like “be accurate”
Test with representative and out-of-scope prompts	Assume permissions work the same in every flow
Document purpose, sources and risks for handover	Keep working in Studio when the job needs code

Golden rule: the easier it is to build an agent, the more important it is that someone owns it, tests it and can switch it off.

What You Can Do Now

Six outcomes — from a studio idea to a governed, handover-ready workflow.

- ✓ Explain agents, automation flows and apps in Rovo Studio
- ✓ Write six-part instructions and ground an agent on chosen sources
- ✓ Connect an agent to a trigger, actions and a human checkpoint
- ✓ Choose the right autonomy level for each task
- ✓ Apply permission, scope and ownership controls
- ✓ Decide when to hand over to engineering, with a handover pack

NEXT: MODULE 10 — CODE-BASED AGENT DEVELOPMENT (COPILOT SDK) & SIMPLE RAG FLOW · 2 HOURS

Build the code-based equivalent: an agent with tools, then ground it with a simple retrieval step.

TODAY'S BUILD PATH



Questions & Discussion

Module 09 — Studio-Style No-Code Agents (Rovo Agent Studio)